

Data Preprocessing, Data Understanding & Feature Engineering

What are the data types in machine learning, and why are they so important? Understanding the different data types is crucial for developing accurate and effective machine learning models. Each data type, whether numerical, categorical, text-based, or visual, presents unique challenges and opportunities in model training and analysis.

What is Data?

Data is a collection of raw facts that can be processed to extract meaningful information. In Machine Learning, data is the foundation for building predictive models.

Types of Data

Understanding the various data types in machine learning is crucial for selecting appropriate methods and achieving effective results. Each data type presents unique characteristics that influence how it is processed, analyzed, and utilized in machine learning models. Below, we explore these data types, categorized by annotation, quantitative and qualitative characteristics, and structure.

- | | |
|-----------------------------|---------------------------------|
| 1. Data types by annotation | 8. Data types by origin |
| 2. Quantitative data | 9. Data types by uniqueness |
| 3. Qualitative data | 10. Data types by accessibility |
| 4. Data types by structure | 11. Data types by quality |
| 5. Data types by source | 12. Data types by lifecycle |
| 6. Data types by timeline | 13. New data types |
| 7. Data types by purpose | |

Quantitative data

Quantitative data is numerical and can be further categorized into continuous, numerical, discrete, and time series data.

- **Continuous data** can take any value within a range, such as temperature readings or height measurements, providing precise information that is critical for various analytical tasks.
- **Numerical data**, which includes both continuous and discrete data, represents quantifiable numbers like age, income, or the number of products sold.
- **Discrete data**, on the other hand, consists of distinct, separate values, such as the number of students in a class. It is easier to categorize and analyze, but handling large categories can be computationally demanding.
- **Time series data**, which is collected at successive points in time, is vital for forecasting and trend analysis, allowing for insights into patterns and changes over time.

Qualitative data

Qualitative data encompasses a range of data types that are not inherently numerical but provide valuable insights.

- **Audio data** includes sound recordings like speech or music, which can be used in applications such as speech recognition and audio classification. Processing audio data requires converting it into a format that can be analyzed, often involving complex transformations.
- **Categorical data** represents distinct categories or groups, such as gender, country, or product type. This type of data is straightforward to interpret, but often needs to be converted into a numerical format for use in machine learning models.

- **Text data**, found in emails, social media posts, or articles, is rich in information and is widely used in natural language processing tasks. However, text data requires extensive preprocessing to extract useful features.
- **Video data**, consisting of sequences of frames, is used in tasks like video classification and object detection. The complexity of video data requires significant computational resources to process. Similarly, image data, such as photographs or medical scans, is widely used in computer vision tasks. This data type provides valuable visual information but requires preprocessing steps like resizing and normalization.
- **3D models and 3D graphics** represent objects in three-dimensional space and are essential in fields like virtual reality, gaming, and 3D printing. Working with 3D data is complex, requiring advanced techniques to manage the added dimension.

Data types by structure

- Structured data is highly organized, typically stored in tables or databases with well-defined schemas, such as SQL databases or spreadsheets. This makes it easy to query and analyze using traditional methods, but it is less flexible in handling unanticipated data types or structures.
- Unstructured data lacks a predefined format, making it more challenging to process. However, it is flexible and can capture a wide range of information, making it invaluable in fields like text analysis and computer vision. Unstructured data often requires advanced techniques to extract meaningful insights and convert them into a more usable form.
- Semi-structured data has some organizational properties but is not fully structured, such as JSON files, XML, or HTML documents. This type of data offers a balance between flexibility and structure, making it easier to work with than unstructured data. However, parsing and querying semi-structured data can be complex, requiring specialized tools and methods.

New data types

Graph data

Graph data represents relationships between entities in the form of nodes (entities) and edges (connections). Examples include social networks, molecular structures, and knowledge graphs.

Graph data is powerful for modeling complex, interconnected systems, enabling applications in recommendation systems, network analysis, and more. However, working with graph data requires specialized algorithms and tools to effectively capture and analyze the relationships between entities.

Sensor data

Sensor data is generated by devices that measure physical properties, such as temperature, humidity, motion, or light. This data is essential for the Internet of Things (IoT), smart devices, and environmental monitoring. Sensor data is typically continuous and time-stamped, making it valuable for real-time analysis and control systems. The challenge with sensor data lies in managing large volumes of streaming data and ensuring its accuracy and reliability.

Cartographic data

Cartographic data includes geographic information, such as maps, satellite imagery, and spatial data. This data is used in applications like geographic information systems (GIS), urban planning, and navigation. Cartographic data provides insights into spatial relationships and geographic trends, but it requires advanced tools and techniques to handle the complexity and scale of spatial information effectively.

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is an important step in data analysis where we explore, summarize, and visualize data to understand its structure, detect patterns, identify anomalies, test assumptions, and check relationships between variables before applying any machine any machine learning or statistical models.

Importance

- Provides a clear understanding of the dataset, including the number of features, data types and data distribution.
- Reveals patterns and relationships between different variables in the data.
- Identifies errors and outliers that may affect analysis.
- Highlights the most important features useful for building models.
- Supports selecting suitable modelling techniques for better results.

Types of Exploratory Data Analysis

1. Univariate Analysis

Univariate analysis studies one variable at a time to understand its characteristics and distribution.

Histograms: Show how data values are distributed.

Box plots: Help detect outliers and show data spread.

Bar charts: Used for categorical variables.

2. Bivariate Analysis

Bivariate analysis examines the relationship between two variables to understand how they interact or influence each other. Common techniques include:

Scatter plots: Show the relationship between two numerical variables.

Correlation coefficient: Measures the strength of the relationship between variables .

Cross-tabulation: Displays the relationship between two categorical variables.

Line graphs: Compare two variables over time to identify trends.

Covariance: Shows how two variables change together.

3. Multivariate Analysis

Multivariate analysis studies three or more variables together to understand complex relationships within the dataset. Common techniques include:

Pair plots: Show relationships between multiple variables at once.

Principal Component Analysis (PCA): Reduces dimensionality while preserving important information.

Spatial analysis: Analyzes geographical patterns using maps and location-based data.

Data Cleaning

Data cleaning is the process of preparing raw data by detecting and correcting errors so it can be effectively used for analysis. It is a foundational step in data preprocessing that ensures datasets are suitable for analytical, statistical and machine learning tasks.

Common Data Anomalies

Data quality issues can arise from human errors, system failures or problems during data collection and integration. Some of the most common data quality challenges include:

Missing values: Incomplete records can reduce statistical power and introduce bias into analysis.

How to Handle:

- Remove rows/columns with many missing values.
- Fill with **Mean** (numerical data).
- Fill with **Median** (if data has outliers or is skewed).
- Fill with **Mode** (categorical data).
- Use **KNN Imputation** or predictive models for better estimation.

Duplicate records: Repeated entries may overrepresent certain observations resulting in skewed outcomes.

How to Handle:

- Identify duplicate rows.
- Remove duplicate records.
- Keep only one unique record.

Incorrect data types: Mismatched formats, such as text stored in numeric fields can cause calculation errors and analysis failures.

How to Handle:

- Convert text to numeric values.
- Convert strings to date/time format.
- Ensure each column has the correct data type.

Outliers and anomalies: Extremely high or low values can distort statistical measures and influence model performance.

How to Handle:

- Detect using **Z-score** or **IQR**.
- Remove erroneous outliers.
- Cap extreme values (Winsorization).
- Transform data (e.g., log transformation) if needed.
- Keep genuine outliers if they represent valid observations.

Inconsistent formats: Variations in date formats, text casing or measurement units can create issues when merging or comparing datasets.

How to Handle:

- Standardize text case (e.g., title case).
- Convert dates to a single format.
- Use consistent units (e.g., all heights in cm).

Spelling and typographical errors: Errors in text fields can lead to incorrect grouping, classification or interpretation of categorical data.

How to Handle:

- Correct using a predefined mapping (dictionary).
- Use fuzzy matching for similar words.
- Validate entries against a list of allowed values.

Feature Engineering

Feature Engineering is the process of selecting, creating or modifying features like input variables or data to help machine learning models learn patterns more effectively. It involves transforming raw data into meaningful inputs that improve model accuracy and performance.

Examples:

- Date $\rightarrow$ Year, Month, Day
- Age $\rightarrow$ Age Group
- BMI = $\text{Weight(kg)}/\text{Height}^2(\text{m}^2)$
- Area = Length $\times$ Width
- Polynomial Features: $x \rightarrow x, x^2, x^3$

Processes Involved in Feature Engineering

Let's see various features involved in feature engineering:



Processes involved in Feature Engineering

1. Feature Creation: Feature creation involves generating new features from domain knowledge or by observing patterns in the data. It can be:

- **Domain-specific:** Created based on industry knowledge like business rules.
- **Data-driven:** Derived by recognizing patterns in data.
- **Synthetic:** Formed by combining existing features.

2. Feature Transformation: Transformation adjusts features to improve model learning:

- **Normalization & Scaling:** Adjust the range of features for consistency.
- **Encoding:** Converts categorical data to numerical form i.e one-hot encoding.

- **Mathematical transformations:** Like logarithmic transformations for skewed data.

3. Feature Extraction: Transform existing features into a lower-dimensional or more informative representation (e.g., PCA).

- **Dimensionality reduction:** Techniques like PCA reduce features while preserving important information.
- **Aggregation & Combination:** Summing or averaging features to simplify the model.

4. Feature Selection: Feature selection involves choosing a subset of relevant features to use:

- **Filter methods:** Based on statistical measures like correlation.
- **Wrapper methods:** Select based on model performance.
- **Embedded methods:** Feature selection integrated within model training.

5. Feature Scaling: Scaling ensures that all features contribute equally to the model:

- **Min-Max scaling:** Rescales values to a fixed range like 0 to 1.
- **Standard scaling:** Standardizes features to have mean 0 and variance 1

Encoding Categorical Variables

Categorical data refers to variables that belong to distinct categories such as labels, names or types. Since most machine learning algorithms require numerical inputs, encoding categorical data to numerical data becomes important. Proper encoding ensures that models can interpret categorical variables effectively, leading to improved predictive accuracy and reduced bias.

Types of Categorical Data

1. Nominal Data: Nominal data consists of categories without any inherent order or ranking. These are simple labels used to classify data.

- **Example:** 'Red', 'Blue', 'Green' (Car Color).
- **Encoding Options:** One-Hot Encoding or Label Encoding, depending on the model's needs.

2. Ordinal Data: Ordinal data includes categories with a defined order or ranking, where the relationship between values is important.

- **Example:** 'Low', 'Medium', 'High' (Car Engine Power).
- **Encoding Options:** Ordinal Encoding.

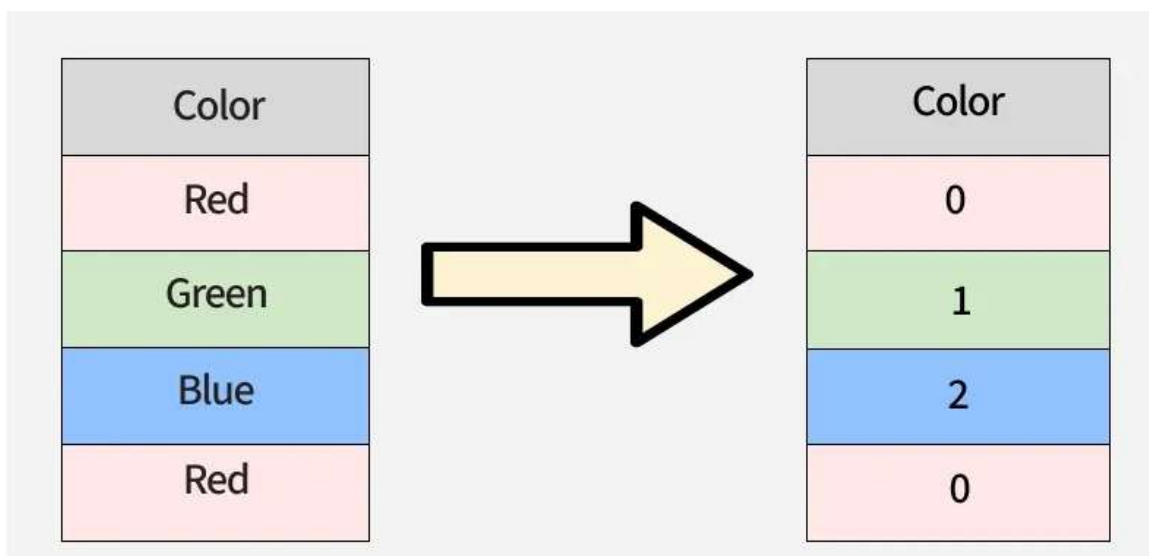
1. Label Encoding

Label Encoding assigns each category a unique integer. It is simple and memory-efficient but may unintentionally imply an order among categories when none exists.

Used in tree-based models like Decision Trees or XGBoost.

Pros: Simple and memory-efficient.

Cons: Introduces implicit order which may be misinterpreted by non-tree models when used with nominal data.



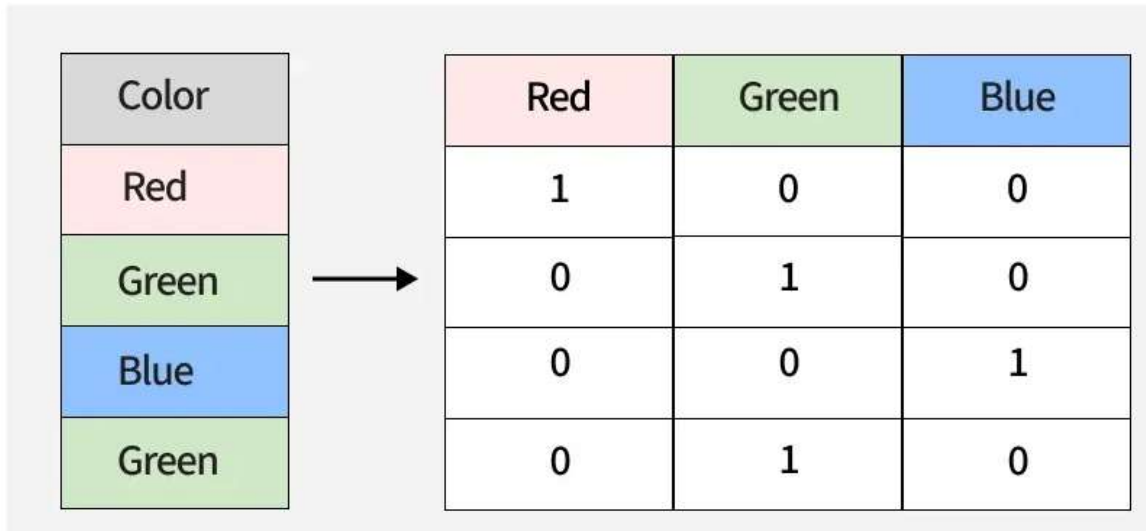
2. One-Hot Encoding

One-Hot Encoding converts categories into binary columns with each column representing one category. It prevents false ordering but can lead to high dimensionality if there are many unique values.

Used in linear models, logistic regression and neural networks.

Pros: Does not assume order; widely supported.

Cons: Can cause high dimensionality and sparse data when feature has many categories.



3. Ordinal Encoding

Ordinal Encoding maps categories to integers while preserving their natural order. This works well for ordered data like ratings but is not suitable for nominal variables.

Used for ordered features like ratings or education levels.

Pros: Maintains order; reduces dimensionality.

Cons: Not suitable for nominal categories.

Original Encoding	Ordinal Encoding
Poor	1
Good	2
Very Good	3
Excellent	4

4. Target Encoding

Target Encoding also known as Mean Encoding is a technique where each category in a feature is replaced by the mean of the target variable for that category.

Useful for high-cardinality features like ZIP codes or product IDs.

Pros: Captures relationship to target variable.

Cons: Risk of overfitting, also must apply smoothing/statistical techniques.

Category	Mean Target (Experience)
Engineer	0.8
Lawyer	0.5
Nurse	0.3

Feature Scaling

Feature Scaling is a technique to standardize the independent features present in the data in a fixed range. It is performed during the data pre-processing.

Methods of Feature Scaling

There are three commonly used methods:

1. Min-Max Scaling (Normalization)
2. Standardization (Z-score Scaling)
3. Robust Scaling

Min-Max Scaling

Min-Max Scaling rescales features by subtracting the minimum value and dividing by the difference between the maximum and minimum values. This usually maps feature values to the range 0 to 1 while preserving the original distribution.

Scaling Formula:

$$X_{\text{scaled}} = \frac{X_i - X_{\min}}{X_{\max} - X_{\min}}$$

Standardization

Standardization scales features by subtracting the mean and dividing by the standard deviation. This transforms the data so that features have zero mean and unit variance, which helps many machine learning models perform better.

Scaling Formula:

$$X_{\text{scaled}} = \frac{X_i - \mu}{\sigma}$$

- where μ = mean, σ = standard deviation.
- Produces features with mean 0 and variance 1.
- Effective for data approximately normally distributed.

Robust Scaling

Robust Scaling scales features using the median and interquartile range (IQR) instead of the mean and standard deviation. This makes it less sensitive to outliers and skewed data, making it suitable for datasets with extreme values or noise.

Scaling Formula:

$$X_{\text{scaled}} = \frac{X_i - X_{\text{median}}}{IQR}$$

Feature Selection

Feature selection is the process of identifying and selecting a subset of the most relevant features from your dataset to use in building a machine learning model.

Why Would You Use Feature Selection?

1. Improve Model Accuracy

Irrelevant or noisy features can confuse the model and reduce performance. By selecting the right subset, models learn patterns more effectively.

2. Reduce Overfitting

Too many features can cause models to memorize noise rather than learn general patterns. Feature selection helps eliminate unnecessary data, making your model more generalizable.

3. Reduce Training Time and Computational Cost

Fewer features → Faster training → Lower memory usage. This is especially important for large datasets or complex algorithms.

4. Improve Model Interpretability

With fewer variables, it becomes easier to:

- Understand how the model makes decisions
- Explain results to stakeholders
- Provide transparency in domains like finance, healthcare, etc.

5. Eliminate Multicollinearity

Highly correlated features can mislead models like linear regression. Feature selection removes redundant features.

6. Helps in Building Simplified, Efficient Models

- Simpler models tend to:
- Be more stable
- Be easier to debug
- Perform better in real-world deployment

Types of Feature Selection Methods

A. Filter Methods

Based on statistical tests.

- Correlation
- Chi-square test
- ANOVA
- Mutual Information

B. Wrapper Methods

Use models to evaluate subsets.

- Forward Selection
- Backward Elimination
- Recursive Feature Elimination (RFE)

C. Embedded Methods

Feature selection happens during model training.

- Lasso (L1 regularization)
- Decision Trees / Random Forest feature importance
- Gradient Boosting methods

Handling Imbalanced Data

Imbalanced data occurs when one class has far more samples than others, causing models to favor the majority class and perform poorly on the minority class. This often results in misleading accuracy, especially in critical applications like fraud detection or medical diagnosis.

- ML models tend to get biased toward the majority class and predict it more frequently.
- Minority class instances may be treated as noise, causing the model to overlook them.
- Accuracy becomes misleading because the model performs well only on the dominant class.
- Skewed decision boundaries lead to poor generalisation and weak performance on minority class predictions.

Techniques:

Oversampling: Duplicates or generates minority class samples to help the model learn more patterns.

Undersampling: Removes majority class samples to balance the dataset and give equal importance to all classes.

Synthetic Minority Oversampling Technique (SMOTE): SMOTE is a data level technique used to handle imbalanced datasets by creating new synthetic samples for the minority class instead of duplicating existing ones. In SMOTE a minority class data point is first selected then its k-nearest neighbors are identified and finally a new synthetic data point is generated between the selected point and one of its neighbors.

ADASYN (Adaptive Synthetic Sampling) : ADASYN (Adaptive Synthetic Sampling Approach) is an oversampling technique used to solve the class imbalance problem in machine learning. It is an improved version of SMOTE because it generates more synthetic

samples for minority class instances that are harder to learn and fewer samples for those that are easier.

Feature Extraction:

- PCA
- TF-IDF (Text)
- Word Embeddings (Introduction)